

Python for Artificial Intelligence

Day 4: Lists & Tuples, In Depth

Month 1: Python Mastery | LLM & Agentic AI Track

Today's Agenda

- Hour 1: Creating, indexing, and slicing lists and tuples
- Hour 2: Mutability and common list methods, tuple use cases
- 15-Minute Break
- Hour 3: Practice set, reverse a list, duplicates, second-largest, tuple unpacking

Day 3 Quick Review (10 mins)

Before we start, try these on your own:

1. Write a for loop that prints the numbers 1 to 10.

2. Write a while loop that counts down from 5 to 1.

3. What does break do inside a loop? What about continue?

4. Swap two variables using tuple unpacking.

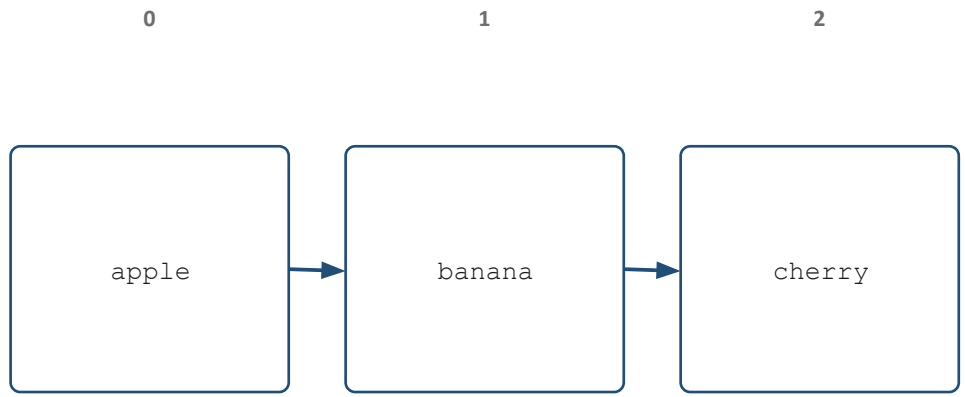
5. Write a one-line FizzBuzz check for a single number n (just the condition logic, no loop).



Hour 1: Creating, Indexing & Slicing

Lists: An Ordered, Changeable Collection

- A list holds multiple values in a single variable, in order
- Lists are mutable, you can add, remove, or change items after creation
- Written with square brackets, items separated by commas
- Items can be any type, and can even be mixed types



index, value pairs, one per round

list: Full Syntax

SYNTAX

```
my_list = [item1, item2, item3]
empty_list = []
```

PARAMETERS

`item1, item2, ...`: Any number of comma-separated values. Types can be mixed.

YOU CAN

✓ Store any mix of types in one list: [1, "two", 3.0, True]

✓ Nest lists inside lists: [[1, 2], [3, 4]]

✓ Create an empty list with [] and add to it later

YOU CANNOT

✗ Use a list as a dictionary key later on, lists are mutable and unhashable (Day 5)

✗ Assume order doesn't matter, a list always remembers insertion order

✗ Access an index that doesn't exist, list[10] on a 3-item list raises IndexError

Returns: *N/A, this is a literal, not a function call.*

Indexing & Slicing: Full Syntax

SYNTAX

```
sequence[index]
sequence[start:stop:step]
```

PARAMETERS

`index`: A single position. 0 is first, -1 is last.

`start`: Where the slice begins, included. Omit to start from the beginning.

`stop`: Where the slice ends, excluded. Omit to go to the end.

`step`: How many items to skip each time. Omit for 1. Use -1 to reverse.

YOU CAN

✓ Use negative numbers anywhere: list[-1], list[:-2]

✓ Omit start, stop, or step entirely: list[:], list[::2]

✓ Slice past the end safely, list[2:100] just stops at the real end, no error

YOU CANNOT

✗ Index past the end, list[100] on a short list raises IndexError, slicing does not

✗ Use a float or string as an index, list[1.5] raises TypeError

✗ Modify a tuple through indexing, tuple[0] = x always raises TypeError

Returns: *A single item for [index]; a new list or tuple (same type as the original) for a slice.*

Creating & Indexing a List

Indexing starts at 0. Negative indexes count from the end.

```
fruits = ["apple", "banana", "cherry", "date"]

print(fruits[0])    # apple
print(fruits[-1])   # date
print(len(fruits))  # 4
```

QUESTION

Think & Practice

Given colors = ["red", "green", "blue", "yellow"], print the second item and the second-to-last item, using indexing.

ANSWER

Answer

```
colors = ["red", "green", "blue", "yellow"]
print(colors[1])
print(colors[-2])
```

Slicing a List

list[start:stop] returns a new list, stop is not included.

```
numbers = [10, 20, 30, 40, 50]

print(numbers[1:3])   # [20, 30]
print(numbers[:2])    # [10, 20]
print(numbers[2:])    # [30, 40, 50]
print(numbers[::-1])  # [50, 40, 30, 20, 10]
```

QUESTION

Think & Practice

Given numbers = [5, 10, 15, 20, 25, 30], use slicing to print only the last three numbers, and separately print every other number starting from the first.

ANSWER

Answer

```
numbers = [5, 10, 15, 20, 25, 30]
print(numbers[-3:])
print(numbers[::2])
```

tuple: Full Syntax

SYNTAX

```
my_tuple = (item1, item2, item3)
single_item = (5,)       # note the comma
empty_tuple = ()
```

PARAMETERS

`item1, item2, ...`: Any number of comma-separated values. The parentheses are optional, Python looks at the commas.

YOU CAN

✓ Create a single-item tuple, but only with a trailing comma: (5,)

✓ Skip the parentheses entirely: point = 4, 7 is still a tuple

✓ Mix types just like a list: ("Ali", 21, 3.7)

YOU CANNOT

✗ Write (5) and expect a tuple, that's just the number 5 in parentheses

✗ Change, add, or remove an item after creation, every mutating list method is missing on purpose

✗ Sort a tuple in place, there is no .sort() method, use sorted(my_tuple) to get a new list instead

Returns: *N/A, this is a literal, not a function call.*

Tuples: An Ordered, Unchangeable Collection

Written with parentheses. Faster than lists, and cannot be modified after creation.

```
point = (4, 7)
colors = ("red", "green", "blue")

print(point[0])     # 4
print(colors[-1])   # blue
# colors[0] = "black"   # this would raise an error
```

Indexing & Slicing a Tuple

Tuples support the same indexing and slicing as lists, just no modification.

```
student = ("Ali", 21, "Computer Science", 3.7)

print(student[1])       # 21
print(student[1:3])     # (21, 'Computer Science')
print(student[-1])      # 3.7
```

QUESTION

Think & Practice

Given person = ("Zara", 28, "Engineer", "Lahore"), print the person's profession using indexing, and print just the first two items using slicing.

ANSWER

Answer

```
person = ("Zara", 28, "Engineer", "Lahore")
print(person[2])
print(person[:2])
```

EXAMPLE 1

Practice: List Basics

Type this code yourself and run it.

```
numbers = [4, 8, 15, 16, 23, 42]
print("First:", numbers[0])
print("Last:", numbers[-1])
print("Middle three:", numbers[1:4])
print("Reversed:", numbers[::-1])
```

QUESTION 1

Think & Practice

Given a list of 10 numbers, print only the first half and only the second half, using slicing.

ANSWER

Answer 1

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
midpoint = len(numbers) // 2

print("First half:", numbers[:midpoint])
print("Second half:", numbers[midpoint:])
```



Hour 2: Mutability, Methods & Tuple Use Cases

append(): Full Syntax

SYNTAX

```
list_name.append(item)
```

PARAMETERS

`item`: The single value to add to the end of the list. Can be any type.

YOU CAN

- ✓ Add any type of value: a number, string, list, or tuple
- ✓ Call it multiple times in a row to add several items one at a time
- ✓ Add a duplicate of a value already in the list

YOU CANNOT

- ✗ Add multiple separate items in one call, that's extend()
- ✗ Insert at a specific position, that's insert()
- ✗ Use the return value, append() always returns None

Returns: *None. The list is modified in place.*

insert(): Full Syntax

SYNTAX

```
list_name.insert(index, item)
```

PARAMETERS

`index`: The position to insert at. Everything from this index onward shifts right.

`item`: The value to insert.

YOU CAN

✓ Use an index larger than the list's length, Python just inserts at the end, no error

✓ Insert at index 0 to add to the front

✓ Insert any type of value, same as append()

YOU CANNOT

✗ Insert multiple items in one call, only one item per insert()

✗ Expect it to overwrite the item at that index, it shifts, it doesn't replace

Returns: *None. The list is modified in place.*

remove(): Full Syntax

SYNTAX

```
list_name.remove(value)
```

PARAMETERS

`value`: The value to search for and delete. The first match is removed.

YOU CAN

✓ Remove any value that exists in the list, by value, not position

✓ Call it again after fixing the error, if the value truly isn't there yet

YOU CANNOT

✗ Remove by index, that's not what remove() does, use pop(index) instead

✗ Remove a value that isn't in the list, this raises ValueError: list.remove(x): x not in list

✗ Remove every matching duplicate at once, only the first match is deleted

Returns: *None. The list is modified in place.*

sort(): Full Syntax

SYNTAX

```
list_name.sort()
list_name.sort(reverse=True)
```

PARAMETERS

`reverse`: Optional. Set to True to sort highest to lowest. Defaults to False.

YOU CAN

✓ Sort numbers, or sort strings alphabetically, each on their own

✓ Sort in descending order with reverse=True

✓ Call it with no arguments for the default ascending order

YOU CANNOT

✗ Sort a list containing mixed incompatible types, [1, "two", 3].sort() raises TypeError

✗ Keep using the old list after x = x.sort(), that reassigns x to None

✗ Sort a tuple, there is no .sort() method on tuples, use sorted(my_tuple) instead

Returns: *None. The list is modified in place.*

Modifying a List

Lists are mutable: append, insert, remove, and sort in place.

```
fruits = ["apple", "banana"]
fruits.append("cherry")
fruits.insert(1, "kiwi")
fruits.remove("apple")
fruits.sort()

print(fruits)
```

QUESTION

Think & Practice

Start with tasks = ["wake up", "eat breakfast"]. Add "go to work" to the end, insert "brush teeth" at index 1, then remove "eat breakfast". Print the final list.

ANSWER

Answer

```
tasks = ["wake up", "eat breakfast"]
tasks.append("go to work")
tasks.insert(1, "brush teeth")
tasks.remove("eat breakfast")
print(tasks)
```

extend(): Full Syntax

SYNTAX

```
list_name.extend(iterable)
```

PARAMETERS

`iterable`: Another list (or any sequence) whose items get added one by one.

YOU CAN

✓ Pass in a list, a tuple, or even a string (it adds each character separately)

✓ Extend a list with itself: numbers.extend(numbers) doubles it

YOU CANNOT

✗ Pass in a single number expecting it to be added as one item, extend(5) raises TypeError

✗ Confuse it with append(): extend([1,2,3]) adds three items, append([1,2,3]) adds one nested list

Returns: *None. The list is modified in place.*

pop(): Full Syntax

SYNTAX

```
list_name.pop()
list_name.pop(index)
```

PARAMETERS

`index`: Optional. Which position to remove and return. Defaults to -1, the last item.

YOU CAN

✓ Call it with no argument to remove and return the last item

✓ Pass a specific index to remove and return that item instead

✓ Store the result, unlike append/insert/remove, pop() actually gives something back

YOU CANNOT

✗ Call it on an empty list, this raises IndexError: pop from empty list

✗ Pass a value instead of an index, pop("apple") does not search by value, that's remove()

Returns: *The removed item, not None.*

count(): Full Syntax

SYNTAX

```
list_name.count(value)
```

PARAMETERS

`value`: The value to count occurrences of.

YOU CAN

✓ Count any type: numbers, strings, even tuples inside the list

✓ Safely count something that isn't there at all, it just returns 0

YOU CANNOT

✗ Expect an error if the value is missing, count() never raises an error, unlike remove()

✗ Count items inside nested lists automatically, it only checks top-level items

Returns: *An integer, how many times the value appears (0 if not found).*

More List Methods

extend(), pop(), and count() round out the everyday toolkit.

```
numbers = [3, 1, 4, 1, 5]
numbers.extend([9, 2, 6])
last = numbers.pop()
how_many_ones = numbers.count(1)

print(numbers)
print("Removed:", last)
print("Count of 1:", how_many_ones)
```

QUESTION

Think & Practice

Given nums = [7, 2, 9, 2, 5, 2], use a list method to count how many times 2 appears, then use another method to remove just the last item and print what was removed.

ANSWER

Answer

```
nums = [7, 2, 9, 2, 5, 2]
print(nums.count(2))

removed = nums.pop()
print("Removed:", removed)
print(nums)
```

Why Would You Ever Use a Tuple?

- Tuples are faster than lists and use less memory, since Python knows they won't change
- Immutability is a feature: a tuple can't be accidentally modified elsewhere in your code
- Functions that return more than one value almost always return a tuple
- Tuples can be used as dictionary keys; lists cannot, because lists are mutable

Tuple Unpacking: Full Syntax

SYNTAX

```
a, b, c = my_tuple
```

PARAMETERS

`a, b, c`: One variable per item in the tuple, in matching order.

YOU CAN

✓ Unpack any sequence this way, not just tuples, lists work too: x, y = [1, 2]

✓ Unpack the result of an expression that produces multiple comma-separated values

✓ Use it inside a for loop to unpack each tuple in a list of tuples, one pass at a time

YOU CANNOT

✗ Use fewer or more variables than there are items, both directions raise ValueError

✗ Skip an item without a placeholder, basic unpacking has no blank slot for that

Returns: *N/A, this is an assignment, not a function call.*

Unpacking a Tuple

Assign every item in a tuple to its own variable, in one line.

```
coordinates = (33.6844, 73.0479, "Islamabad")
latitude, longitude, city = coordinates

print(f"{city} is at ({latitude}, {longitude})")
```

QUESTION

Think & Practice

Given student = ("Bilal", "Math", 3.8), unpack it into three variables in one line and print a sentence using all three.

ANSWER

Answer

```
student = ("Bilal", "Math", 3.8)
name, subject, gpa = student

print(f"{name} studies {subject} with a GPA of {gpa}")
```

Lists vs. Tuples

Same indexing and slicing rules, different mutability and typical use.

```
# List: things that change over the program's life
shopping_cart = ["milk", "eggs"]
shopping_cart.append("bread")

# Tuple: a fixed record that shouldn't change
coordinates = (33.6844, 73.0479)   # latitude, longitude
```

EXAMPLE 1

Practice: Shopping Cart Manager

Type this code yourself and run it.

```
cart = ["milk", "eggs", "bread"]
cart.append("butter")
cart.remove("eggs")
cart.sort()

print("Final cart:", cart)
print("Items in cart:", len(cart))
```

QUESTION 1

Think & Practice

Given a list of temperatures, compute the minimum, maximum, and average in one line, then unpack all three into separate variables.

ANSWER

Answer 1

```
temps = [21, 26, 19, 30, 24]
low, high, avg = min(temps), max(temps), sum(temps) / len(temps)

print(f"Low: {low}, High: {high}, Avg: {avg}")
```



15-Minute Break

Stretch, grab a coffee. We resume with Hour 3: Practice Set.



Hour 3: Practice Set

EXAMPLE 1, INTERVIEW FAVORITE

Practice: Reverse a List (Without reverse())

Type this code yourself and run it.

```
numbers = [1, 2, 3, 4, 5]
reversed_list = []

for n in numbers:
    reversed_list.insert(0, n)

print(reversed_list)
```

EXAMPLE 2, INTERVIEW FAVORITE

Practice: Find Duplicates in a List

Type this code yourself and run it.

```
numbers = [1, 2, 3, 2, 4, 5, 1, 6]
seen = []
duplicates = []

for n in numbers:
    if n in seen and n not in duplicates:
        duplicates.append(n)
    seen.append(n)

print(duplicates)
```

sorted(): Full Syntax

SYNTAX

```
sorted(iterable)
sorted(iterable, reverse=True)
```

PARAMETERS

`iterable`: The list, tuple, or other sequence to sort.

`reverse`: Optional. True for descending order. Defaults to False.

YOU CAN

✓ Sort a list without changing the original, unlike .sort()

✓ Sort a tuple directly, sorted() always returns a list

✓ Chain it with indexing to grab a value, e.g. sorted(nums, reverse=True)[1] for second-largest

YOU CANNOT

✗ Modify the original list, sorted() always returns a brand-new list

✗ Sort mixed incompatible types, same TypeError risk as .sort()

Returns: *A new, sorted list. The original sequence is never changed.*

QUESTION 1, INTERVIEW STYLE

Think & Practice

Given a list of numbers, find the second-largest number without sorting the whole list twice or using sorted()[-2].

ANSWER

Answer 1

```
numbers = [12, 45, 2, 41, 31, 10]
sorted_numbers = sorted(numbers, reverse=True)
print("Second largest:", sorted_numbers[1])
```

QUESTION 2, HARDER

Think & Practice (Harder)

Given students = [("Ali", 78), ("Sara", 92), ("Ahmed", 85)],

loop through the list and print each name with their score,

unpacking each tuple directly inside the for loop.

ANSWER

Answer 2

```
students = [("Ali", 78), ("Sara", 92), ("Ahmed", 85)]

for name, score in students:
    print(f"{name}: {score}")
```

Homework: Practice for Interviews

Try these on your own before next class:

1. Write a program that removes duplicate items from a list while keeping the original order.
2. Write code that finds and prints the most frequent item in a list.
3. Write code that merges two sorted lists into one sorted list, without using sort().
4. Given a list of (name, score) tuples, find the tuple with the highest score.
5. Rotate a list to the left by one position, using only indexing and slicing.

Key Takeaways

- Lists are ordered and mutable; tuples are ordered and immutable
- Slicing works the same way on both: list[start:stop], with a negative step to reverse
- Tuples show up constantly as multiple return values from functions
- Interview-style list problems reward knowing sets and dictionaries too, not just loops

Coming Up: Day 5

- Dictionaries & Sets, plus the Week 1 Skill Checkpoint
- Key-value pairs, creating and accessing dictionaries, .get(), .items()
- Modifying dictionaries; set operations (union, intersection, difference)
- Weekly Skill Checkpoint: "I can confidently manipulate all of Python's core data structures"